

SIMATS ENGINEERING
CO1-ASSESSMENT TOOL 1
Analytical Problem Solving

COURSE CODE: MLA0403

COURSE NAME: Deep Learning

FACULTY: Dr.Arul Raj A.M

Duration: 30 minutes | Marks: 25

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

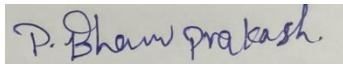
CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Code of Conduct:

I P.Bhanu Prakash / 192425263 (Name/ Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: P.Bhanu Prakash

QUESTIONS: 05 | Total Marks: 25 | Analytical Problem Solving

Question 1: Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question: Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

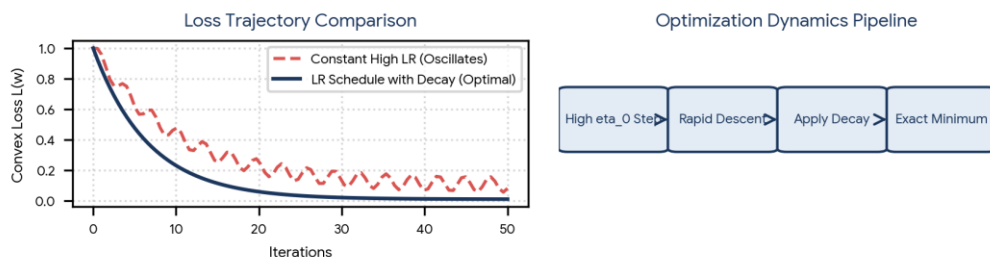
1. Analysis of High Initial Learning Rate with Decay on Convergence

In convex optimization, the gradient descent update equation is $w_{t+1} = w_t - \eta_t \cdot \text{grad } L(w_t)$. Starting with a high initial learning rate (η_0) allows the algorithm to take large parameter steps across flat regions of the loss surface, rapidly traversing long distances toward the global minimum. As training progresses, applying learning rate decay ($\eta_t = \eta_0 / (1 + \text{decay} \cdot t)$ or exponential decay) reduces step sizes, preventing oscillation around the minimum and ensuring asymptotic convergence directly into the optimal point w^* .

2. Conditions Where Decay Outperforms Constant Learning Rate

- **Fast Early Traversal vs. Late Stability:** A small constant learning rate guarantees convergence but takes an extremely high number of iterations. Conversely, a large constant learning rate reaches the neighborhood quickly but bounces indefinitely around the minimum due to residual gradient magnitudes. Learning rate decay achieves both high initial speed and fine-grained late convergence.
- **Ill-Conditioned Convex Surfaces:** In elongated elliptical loss contours (high condition number of the Hessian matrix), large initial steps rapidly reduce major loss components, while late decay prevents divergence along steep orthogonal dimensions.

3. Convergence Dynamics & Decay Pipeline



4. Associated Risks and Mathematical Justification

- **Overshooting and Divergence Risk:** If $\eta_0 > 2 / L$ (where L is the Lipschitz constant of the gradient $\text{grad } L$), initial updates diverge instantly, sending parameters to infinity.
- **Premature Learning Rate Decay:** If decay is too aggressive, step sizes collapse to near-zero before reaching the minimum neighborhood, trapping the model at a sub-optimal point.

5. Conclusion

A high initial learning rate with decay balances optimization speed and asymptotic precision. Provided η_0 complies with Lipschitz stability thresholds, decay guarantees faster convergence to the exact global minimum than any single fixed learning rate.

Question 2: Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2 .

Question: Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

What does this imply about the robustness of MLE?

1. Mathematical Derivation of MLE for Gaussian Mean (μ)

Let $X = \{x_1, x_2, \dots, x_n\}$ be n i.i.d. random variables drawn from $N(\mu, \sigma^2)$. The likelihood function $L(\mu)$ is the product of individual probability density functions:

$$L(\mu) = \prod_{i=1}^n \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right) \right]$$

Taking the natural logarithm yields the log-likelihood function $\log L(\mu)$:

$$\log L(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \sum_{i=1}^n \left[\frac{(x_i - \mu)^2}{2\sigma^2} \right]$$

To find the maximum likelihood estimate $\mu_{\text{hat_MLE}}$, compute the first derivative with respect to μ and set it to zero:

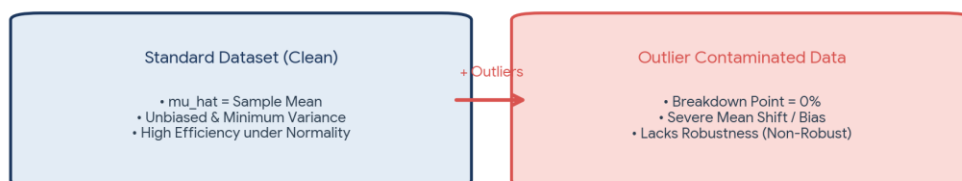
$$\frac{d(\log L(\mu))}{d(\mu)} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$$

$$\Rightarrow \sum_{i=1}^n x_i - n\mu = 0 \Rightarrow \mu_{\text{hat_MLE}} = \frac{1}{n} \sum_{i=1}^n x_i \text{ (The Sample Mean)}$$

2. Analytical Explanation under Special Conditions

- **Impact of Outliers:** If an extreme outlier x_{outlier} is added, $\mu_{\text{hat_MLE}} = (1/(n+1)) * (\sum x_i + x_{\text{outlier}})$. Because the sample mean weights all data points equally, a single outlier can shift $\mu_{\text{hat_MLE}}$ arbitrarily far, causing severe estimation bias.
- **Impact of Increasing Sample Size (Consistency):** By the Strong Law of Large Numbers, as $n \rightarrow$ infinity, $\mu_{\text{hat_MLE}}$ converges almost surely to the true mean μ_{true} . Furthermore, $\text{Var}(\mu_{\text{hat_MLE}}) = \sigma^2 / n \rightarrow 0$, proving MLE is an unbiased and consistent estimator.

3. Robustness Analysis Schematic



4. Implication on MLE Robustness

This analytical breakdown proves that Gaussian MLE has a breakdown point of 0%, meaning even a single corrupted sample can destroy the estimator. While MLE is asymptotically optimal (attaining the Cramér-Rao lower bound), it is not robust to distributional heavy tails or corrupted observations.

Question 3: Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question: Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

1. Analysis of High-Dimensional / Small-Sample Challenges ($p \gg n$)

With $p = 10,000$ features and $n = 500$ samples, the problem is severely underdetermined ($p \gg n$). In this regime, the feature space is extremely sparse, creating infinite parameter configurations that perfectly separate training data. This causes extreme overfitting, collinearity among features, noisy covariance estimates, and high test error variance.

2. Proposed Algorithm Pipeline Strategy

- **Step 1: Data Preprocessing & Variance Thresholding:** Apply Z-score standardization and drop features with near-zero variance across all 500 samples, removing uninformative noise attributes.
- **Step 2: Dimensionality Reduction via PCA:** Project the remaining high-dimensional space onto a lower-dimensional subspace using Principal Component Analysis (PCA) or Linear Discriminant Analysis (LDA) retaining 95% explained variance.
- **Step 3: Feature Selection via Sparse Regularization (L1 Lasso):** Utilize an L1-penalized Logistic Regression model: $\text{Loss} = L(w) + \lambda \|w\|_1$. The L1 norm forces uninformative feature weights to exactly zero, isolating the true predictive signal.
- **Step 4: Regularized Classifier & Nested Cross-Validation:** Train a Ridge/Support Vector Machine (SVM) classifier combined with 5-fold cross-validation to select hyperparameter λ without data leakage.

3. End-to-End Dimensionality Mitigation Pipeline



4. How the Approach Mitigates Overfitting

By pruning redundant dimensions from 10,000 down to ≤ 50 intrinsic features, the effective degrees of freedom match the 500 sample limit ($n > p_{\text{effective}}$). Combining L1 sparsity with L2 weight shrinkage bounds the hypothesis space capacity, forcing the model to learn true statistical patterns rather than memorizing noise.

Question 4: Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question: Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

1. Analytical Explanation of Representational Power

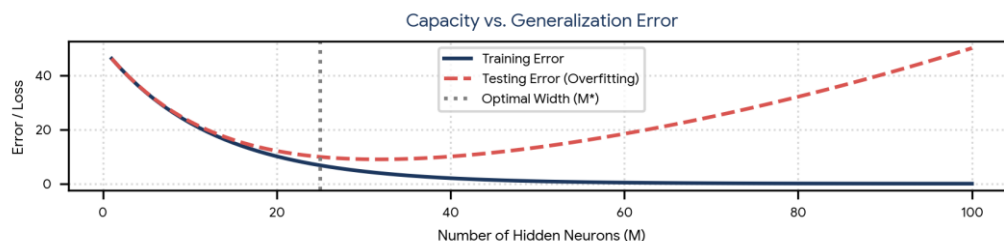
A single hidden layer MLP with sigmoid activation functions is expressed mathematically as $f(x) = \sum_{j=1}^M v_j * \sigma(w_j^T x + b_j)$. By the Universal Approximation Theorem (Cybenko, 1989), any continuous function on compact subsets of $\mathbb{R}^n$ can be approximated to arbitrary accuracy $\epsilon > 0$ by expanding the number of hidden units M .

Each hidden neuron acts as a linear decision boundary smoothed by $\sigma(z)$. Increasing M increases the number of basis functions, allowing the network to synthesize complex, non-linear decision surfaces and capture intricate feature interactions.

2. Why More Neurons Do Not Guarantee Better Performance

- **VC-Dimension & Overfitting:** As M increases, the Vapnik-Chervonenkis (VC) dimension and hypothesis capacity explode. The model fits noise and random fluctuations in the training set, causing zero training loss but huge test error.
- **Vanishing Gradients:** Sigmoid derivatives $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ peak at 0.25. In wide networks without proper weight initialization, gradients quickly saturate, leading to poor parameter optimization.

3. Model Capacity & Generalization Curve



4. Generalization Trade-Off Analysis

Optimal generalization occurs at an intermediate width M^* where the bias-variance trade-off is balanced. Beyond M^* , excess parameters increase variance, degrading generalization performance unless regularized by L2 weight decay or dropout.

Question 5: Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question: Analyze how the curse of dimensionality impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

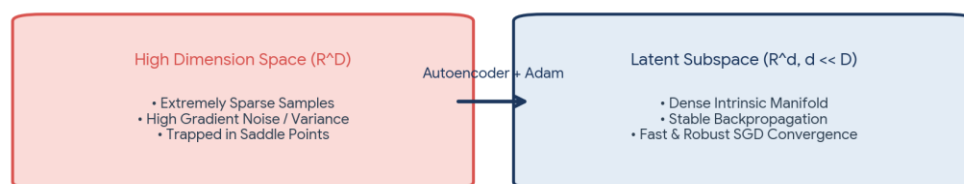
1. Impact Analysis of the Curse of Dimensionality (High D)

- **Impact on Backpropagation Gradient Updates:** In high dimensions, volume grows exponentially (V proportional to r^D), causing inputs to become equidistant and sparse. Gradients $\text{grad}_W L = \Delta * x^T$ become dominated by uninformative noise, leading to gradient vanishing or exploding across deep layers.
- **Impact on SGD Convergence Speed:** High dimensionality introduces non-convex loss surfaces filled with saddle points and ill-conditioned Hessians. Stochastic mini-batch gradients exhibit extreme variance, causing SGD updates to bounce wildly in suboptimal directions and slowing convergence.
- **Impact on Model Generalization:** Data sparsity means sample density drops to near zero. Deep networks memorize training configurations without finding generalizable boundaries, inflating test variance.

2. Proposed Analytical Techniques to Overcome Issues

- **Technique 1: Mini-Batch SGD with Momentum / Adam Optimization:** Replace vanilla SGD with Momentum or Adam updates: $m_t = \beta_1 * m_{t-1} + (1-\beta_1)*g_t$. Momentum averages past gradients, dampening high-dimensional stochastic noise and accelerating trajectory along consistent curvature directions.
- **Technique 2: Dimensionality Reduction via Autoencoders or PCA:** Project inputs x in R^D to z in R^d ($d \ll D$) using an Autoencoder minimizing reconstruction error $\|x - \text{Dec}(\text{Enc}(x))\|^2$. This preserves essential manifold structure while removing noisy dimensions.

3. High-Dimensional Optimization Resolution



3. Conclusion

Combining Adaptive Moment Estimation (Adam) with Autoencoder feature compression addresses the curse of dimensionality, stabilizing backpropagation gradients and accelerating SGD convergence.